OsdagBridge

LaTeX Report Generator — Refactoring & Enhancements

Technical Changes Document

Author	Vivek Manik Warkad (IIT Kanpur)
Repository	`https://github.com/VivekWar/OsdagBridge` (fork of `Nidhikhare12/OsdagBridge`, branch `dev`)
Scope	`src/osdagbridge/core/reports/` (plus the quantity generator in `core/boq/`)
Deliverables	`Report_Before.pdf`, `Report_After.pdf`, this document, source code, unit tests, demo video
Test design	Plate girder highway bridge, 30 m span, 10.5 m carriageway, 5 girders, Pune (Maharashtra), all eight IRC vehicles selected — identical inputs for the "before" and "after" report

1 Summary

The report generator of OsdagBridge assembled the design report from large raw-LaTeX string templates. Formatting values (column widths in cm, `\vspace`, fonts, colours) were repeated in every chapter file, tables that ran over a page lost their header, and row-spanning labels could end up in the footer. The generator was refactored so that

- **every** table is a PyLaTeX `LongTable` produced by *one* builder — caption and column headers repeat on every page a table runs onto;
- nothing can overlap the footer any more: row groups are unbreakable, the page geometry reserves the footer space, and page-bottom thresholds decide when a block starts on a new page;
- vehicle and footpath live loads are reported in separate tables with unit columns, listing exactly the vehicles ticked in the *Loading* tab;
- Section 5.5 shows utilization-ratio bar charts with a red dashed line at UR = 1.0, Chapter 7 shows material-quantity charts — both generated with matplotlib during the report build and embedded with PyLaTeX `Figure`;
- `styles.py` is the single source of truth for all formatting; chapter files contain content only (enforced by a unit test).

Table 1 maps each item of the task to the place where it is implemented. The numerical content of the report is unchanged: the same stored design results are read, only their presentation differs (the few intentional content changes are listed in Section 7).

Table 1: Task items and where they are implemented

Task item	What was done	Main code
1. Repeated table headers	All 61 tables are `longtable`s with `\endfirsthead` / `\endhead`; a "(continued)" caption and the (one- or two-level) column header repeat on each page, "Continued on next page" closes a broken page.	`tables.py`: `ReportLongTable`, `table()`
2. Layout, footer overlap, spacing	Row groups (e.g. all rows of "G4") are unbreakable; table notes are the last row of their table; bottom margin and footskip enlarged; penalty-free page-bottom thresholds for tables, figures and headings; short tables are never split; widow/orphan control for table rows.	`tables.py`: `_body_rows()`; `styles.py`: `PAGE`, `SPACING`, `TABLE`

Table 1: Task items and where they are implemented (continued)

Task item	What was done	Main code
3. Load & geometry tables	Separate *Vehicle Live Loads* table (Vehicle, Total Load, Impact Factor, Braking Load: Considered? / Value / Eccentricity), *Braking and Centrifugal Forces* table and *Footpath Live Load* table; unit columns in all load and geometry tables; vehicles taken from the Loading-tab check boxes.	`live_loads.py`, `chap3.py`, `chap2.py`
4. UR bar charts (5.5)	Two charts: governing UR per element (girders, deck slab, cross bracing, end diaphragms) and UR of every summary check; red dashed line at UR = 1.0, failing bars hatched red.	`charts.py`, `design_summary.py`, `chap5.py`
5. Quantity charts (Ch. 7)	Structural steel tonnage by component; deck concrete volume (m^3) against reinforcement weight (MT) on two axes. Images are written to the temporary build directory and embedded with PyLaTeX `Figure`.	`charts.py`, `chap7.py`, `figures.py`, `boq_generator.py`
6. Central style module	`styles.py`: page, colours, fonts, spacing, table metrics, named column layouts, header/footer, figure widths, chart theme, units. Chapters refer to layouts and widths *by name*.	`styles.py`

2 Before and after

The comparison pages below come from `Report_Before.pdf` (upstream `dev`, commit `d74f56d`) and `Report_After.pdf`, both generated through the desktop application for the same design.

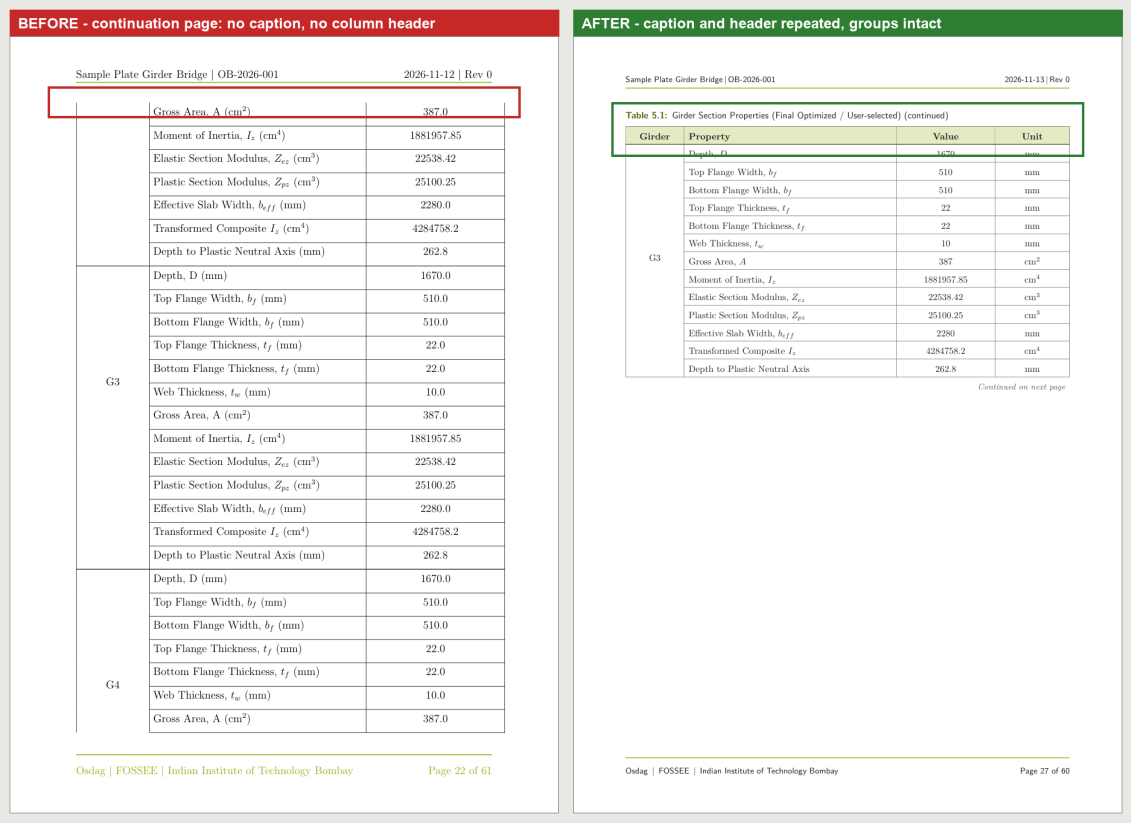


Figure 1: Repeated headers. Before (left): the continuation page of the girder property table starts without caption or column header, and the girder label has lost its rows. After (right): "Table 5.1 (continued)" with the header row, and each girder group is complete on its page.

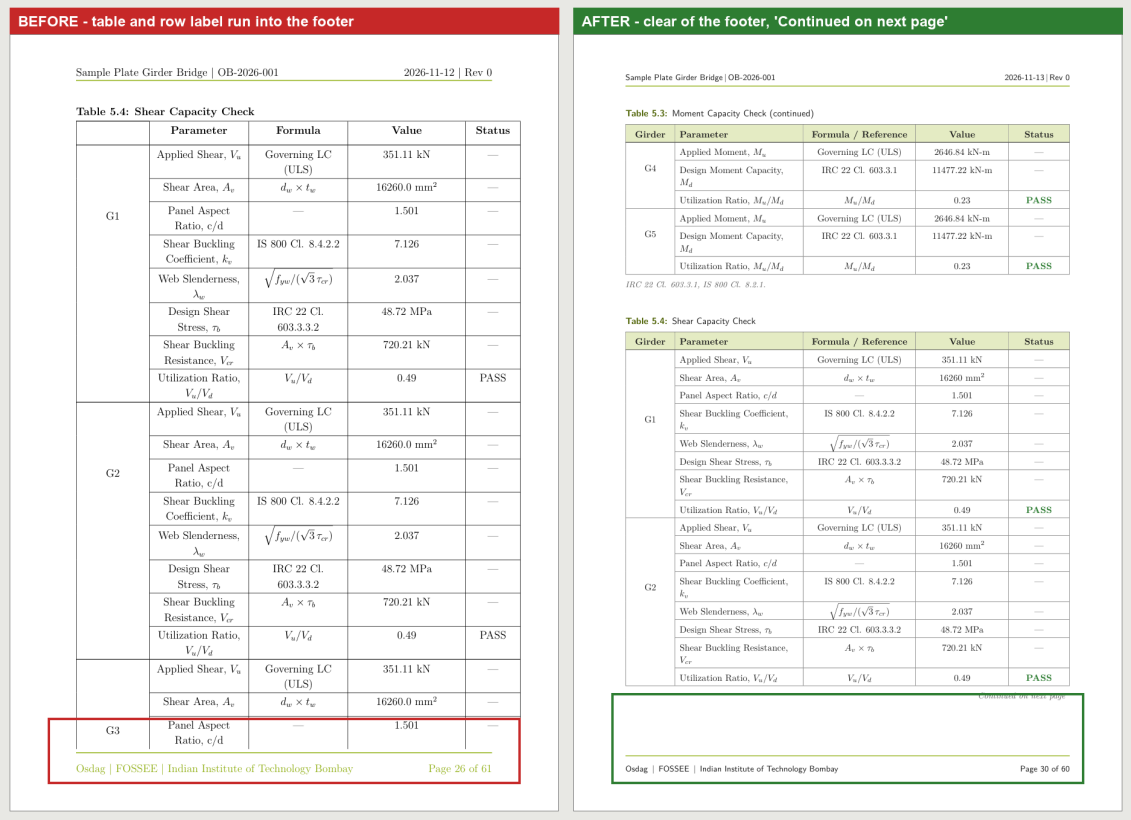


Figure 2: Footer overlap. Before (left): the table runs to the footer rule and the row-spanning label "G3" sits on it. After (right): groups are kept together, the table ends above the footer with a "Continued on next page" line.

BEFORE - merged live load table, empty cells

Sample Plate Girder Bridge | OB-2026-001 2026-11-12 | Rev 0

3 Loads and Load Combinations

This section summarizes all loads applied to the bridge and the load combinations considered for analysis and design.

Table 3.1: Dead Load – Self Weight

parameter	value
Steel Self-Weight Applied	76.4 kN/m^3
Concrete Deck Weight	24.5 kN/m^3
Self-Weight Factor	1.0

Table 3.2: Dead Load for Surfacing (DW)

parameter	value
Wearing Course Load	Concrete x 50.0
Additional SIDL (Crash Barrier)	7.557 kN/m per barrier
Railing Load	1.4715 kN/m*

Table 3.3: Live Loads (LL)

parameter	value
Vehicles Considered	Class A, Class 70R (Wheeled), Class 70R (Tracked), Class AA (Wheeled), Class AA (Tracked), Class SV, Class 70R (Bogie), Class Fatigue
Impact Factor (IRC 6)	Class A: 1.207, Class AA/70R: 1.100
Braking Load (IRC 6)	N/A
Footpath Live Load (if applicable)	

** Software default value*

Osdag | FOSSEE | Indian Institute of Technology Bombay Page 12 of 61

AFTER - separate vehicle / braking / footpath tables

Sample Plate Girder Bridge | OB-2026-001 2026-11-13 | Rev 0

Table 3.3: Vehicle Live Loads (LL) — IRC 6

Vehicle	Total Load (kN)	Impact Factor	Braking Load		
			Considered?	Value (kN)	Eccentricity (m)
Class A	543.5	1.207	Yes	108.7	1.20
Class AA Wheeled	392.4	1.207	Yes	78.5	1.20
Class AA Tracked	686.7	1.100	Yes	137.3	1.20
Class 70R Wheeled	981.0	1.207	Yes	196.2	1.20
Class 70R Tracked	686.7	1.100	Yes	137.3	1.20
Class 70R Bogie	392.4	1.207	Yes	78.5	1.20
Class SV	3776.9	1.000	Yes	755.4	1.20
Class Fatigue	392.4	1.207	Yes	78.5	1.20

Vehicles are those selected under Additional Inputs → Loading → Live Load. Total load is that of one vehicle / train (IRC 6 Cl. 204). Impact factor = 1 + I for steel bridges (IRC 6 Cl. 208): Class A and fatigue vehicle $I = 9/(13.5 + L)$; tracked vehicles $I = 0.10$; wheeled Class AA / 70R $I = 0.25$ up to 23 m span, $9/(13.5 + L)$ beyond; no impact for Class SV (Cl. 204.5). Braking value = 20% of the vehicle load (first train, IRC 6 Cl. 211.2), acting at the stated eccentricity above the top of the deck.

Table 3.4: Live Load — Braking and Centrifugal Forces

Parameter	Value	Unit	Reference / Remarks
No. of Design Traffic Lanes	—	—	IRC 6 Table 6
Braking Force — Governing Vehicle (Class SV)	755.4	kN	20% of first train, IRC 6 Cl. 211.2(a)
Braking Force — Lanes in Excess of Two	N/A	kN	5% of the load on those lanes, IRC 6 Cl. 211.2(b)
Design Braking Force (longitudinal)	755.4	kN	Applied along the roadway
Braking Force — Height above Deck	1.20	m	IRC 6 Cl. 211.3
Centrifugal Force	N/A	kN	IRC 6 Cl. 212: $C = WV^2/(127R)$; not applicable to a straight alignment

Osdag | FOSSEE | Indian Institute of Technology Bombay Page 14 of 60

Figure 3: Live loads. Before (left): one merged table, empty footpath and braking cells. After (right): separate vehicle table (two-level *Braking Load* header, unit columns), braking / centrifugal forces and footpath load.

NEW - Section 5.5 utilization ratio charts

Sample Plate Girder Bridge | OB-2026-001 2026-11-13 | Rev 0

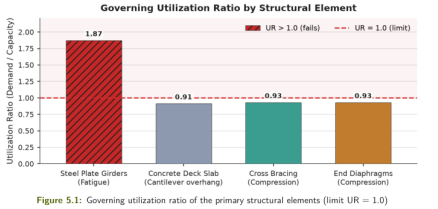


Figure 5.1: Governing utilization ratio of the primary structural elements (limit UR = 1.0)

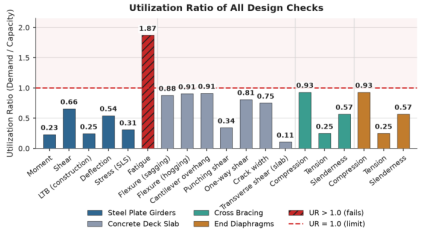


Figure 5.2: Utilization ratio of all summary design checks, grouped by structural element (limit UR = 1.0)

Osdag | FOSSEE | Indian Institute of Technology Bombay Page 46 of 60

NEW - Chapter 7 material quantity charts

Sample Plate Girder Bridge | OB-2026-001 2026-11-13 | Rev 0

Table 7.2: Quantity Summary by Material

Material / Component	Quantity	Unit	Share of Structural Steel
Structural steel			
Plate Girders	41.12	MT	95.7 %
Cross Bracing (chords + diagonals)	0.66	MT	1.5 %
End Diaphragms	1.19	MT	2.8 %
Total Structural Steel	42.97	MT	100.0 %
Deck slab			
Concrete Volume	85.50	m^3	—
Reinforcement Steel	10.26	MT	—
Reinforcement Content	120	kg/m^3	—

7.2 Material Quantity Charts

The charts below are generated from the quantities of the bill of materials.

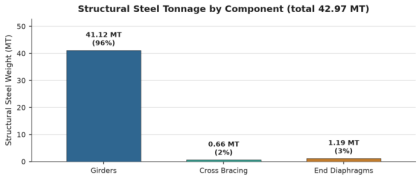


Figure 7.1: Structural steel tonnage by component (girders, cross bracing, end diaphragms)

Osdag | FOSSEE | Indian Institute of Technology Bombay Page 54 of 60

Figure 4: New charts. Left: Section 5.5 utilization ratios with the UR = 1.0 limit line. Right: Chapter 7 quantity summary and structural steel tonnage.

3 Files altered

All paths are relative to `src/osdagbridge/`. "+ / −" are added / removed lines against upstream `dev`.

Table 2: Files added or changed

File	Status	+ / −	Role
`core/reports/styles.py`	new	897 / 0	Single source of truth for all formatting; emits the LaTeX preamble.
`core/reports/tables.py`	new	440 / 0	Table builder on PyLaTeX `LongTable`: repeated headers, groups, bands, notes, keep-together logic.
`core/reports/figures.py`	new	113 / 0	Figure builder on PyLaTeX `Figure`; relative image paths.
`core/reports/charts.py`	new	286 / 0	matplotlib bar charts (UR and material quantities).
`core/reports/live_loads.py`	new	232 / 0	Data for the live-load tables (selected vehicles, impact, braking, footpath).
`core/reports/design_summary.py`	new	339 / 0	Governing checks per element; feeds the 5.5 table, the UR charts and the executive summary.
`core/reports/data_bridge.py`	new	146 / 0	Read-only access to stored results (moved out of `report_generator.py`, returns numbers instead of LaTeX).
`core/reports/report_generator.py`	changed	166 / 489	Orchestration: PyLaTeX `Document`, title page, TOC, chapter assembly, pdflatex passes. Public API unchanged.
`core/reports/chap1.py` ... `chap9.py`, `executive_summary.py`	changed	1918 / 2754	Chapter content rewritten on the builders; no formatting values left.
`core/reports/report_utils.py`	changed	101 / 53	Escaping (incl. Unicode from check names), number formatting, mm conversion.
`core/boq/boq_generator.py`	changed	33 / 0	End-diaphragm steel quantity (needed for the tonnage chart).
`tests/unit/test_report_layout.py`	new	315 / 0	29 unit tests of the layout system.
`tools/report_demo.py`, `tools/render_report.py`	new	381 / 0	Reproducible generation of the before/after reports through the real GUI; fast re-rendering from a stored payload.
`pyproject.toml`, `requirements.txt`, `environment.yml`	changed	6 / 0	`pylatex` added as a dependency.

4 Structure of the style module

`styles.py` uses only the Python standard library. Each group of settings is a frozen dataclass with one module-level instance; functions at the end of the module turn the values into LaTeX.

Table 3: Contents of `styles.py`

Object	Defines
`PAGE`	Document class, paper, font size, margins, head/foot distances, line spacing. `geometry_options()` is passed to PyLaTeX's `Document`.

Object	Defines
`COLORS`	One palette for LaTeX *and* matplotlib (brand green, text, table fills, status colours, chart series). Attribute name = LaTeX colour name; `COLORS.mpl(name)` gives the matplotlib value.
`FONTS`	Font switches for headings, tables (normal / compact), captions, notes, running heads.
`SPACING`	Space around blocks and headings, and the page-bottom thresholds (minimum space required to start a table, figure, heading).
`TABLE`	Cell padding, row stretch, rule widths, header/band fills, "continued" wording, keep-together switches, widow/orphan limits, size-estimate metrics.
`TABLE_LAYOUTS`	35 named column layouts, e.g. `"vehicle_live_load"`: alignment + *relative* width per column. The builder normalises the weights, so every table spans exactly the text width whatever the margins are.
`HEADER_FOOTER`	Running header/footer texts, rule colour and width, the software-default note.
`FIGURE`	Named figure widths (`full`, `wide`, `medium`, `chart`, `third`), maximum height, float placement, logo sizes.
`CHART`	matplotlib theme: sizes, fonts, bar look, element/material colours, threshold value, colour, dash style and label.
`UNITS`, `TEXT`	Unit strings (kN-m, kN/m^2, ...) and fixed wording (PASS / FAIL, N/A, column titles).
`latex_packages()`, `document_arguments()`, `preamble_tex()`	The package list, the keyword arguments of `pylatex.Document` and the complete style preamble (colour definitions, column types, semantic macros, caption / heading / TOC / list set-up, header and footer).
`status_tex()`, `pass_fail()`, `ratio_tex()`	Semantic helpers: coloured PASS / FAIL / WARN cells, highlighted ratios above 1.0.

How chapters use it. A chapter passes content and *names*:

```
T.table("Vehicle Live Loads (LL) --- IRC 6",          # caption
        "vehicle_live_load",                          # layout name from styles.TABLE_LAYOUTS
        ("Vehicle", "Total Load (kN)", "Impact Factor",
         T.Span("Braking Load", ("Considered?", "Value (kN)", "Eccentricity (m)"))),
        rows, note="...", label="tab:vehicle-live-loads")

F.figure(path, "Governing utilization ratio ...", width="chart")   # width name from styles.FIGURE
```

Changing a colour, the table padding, the margins or a column width is a one-line edit in `styles.py`; no chapter file has to be touched. The unit test `test_chapter_files_contain_no_formatting_rules` fails if a chapter file contains `\hline`, `\vspace`, `\textcolor`, `\begin{longtable}`, explicit lengths and similar markup.

5 Key code and LaTeX changes

5.1 Repeated headers (`tables.py`)

`ReportLongTable` subclasses PyLaTeX's `LongTable` and adds what PyLaTeX does not expose: `\endfirsthead`, captions inside the table, rows ending in `\\*` and break-free rules. `table()` writes the caption and header twice — once for the first page, once for all following pages:

```
\begin{longtable}{|C{0.12\rptTableWidth}|L{0.50\rptTableWidth}|...|}
\caption{Girder Section Properties}\label{tab:girder-section-properties}\\ \hline
\rowcolor{rptHeadBg}\rpthead{Girder} & \rpthead{Property} & ... \\ \hline
```

```
\endfirsthead
\caption[]{Girder Section Properties (continued)}\\ \hline      % no second table number
\rowcolor{rptHeadBg}\rpthead{Girder} & \rpthead{Property} & ... \\ \hline
\endhead
\rptcontinued{4} \\                                             % "Continued on next page"
\endfoot
\endlastfoot
```

Two-level headers (`T.Span`) are built with `\multicolumn`; single cells that span both header rows use a negative `\multirow` so the row fill does not paint over them, and `\hhline` draws the rule between the two levels.

5.2 Footer overlap: root causes and fixes

Three independent causes were found in the original output.

1. **Row groups were breakable.** A girder label was a `\multirow` over up to 13 rows. When the page broke inside the group, the label was typeset relative to rows that were no longer on the page and landed in the footer. Now every row of a group except the last ends in `\\*`. That alone is not enough: `\hline` contains a deliberate break opportunity and `\cline` is followed by a breakable skip. Rules inside a group are therefore drawn with `\hhline` followed by an explicit `\noalign{\nobreak}`:

   ```
   \multirow{13}{=}{\centering G4} & Depth, $D$ & 1670 & mm \\*
   \hhline{|~|-|-|-|}\noalign{\nobreak}
    & Top Flange Width, $b_f$ & 510 & mm \\*
   \hhline{|~|-|-|-|}\noalign{\nobreak}
    ...
   ```

2. **The footer was pulled into the text area.** The footer rule was raised with `\vspace{-20pt}` to make room for the "* Software default value" note, inside a default 30 pt footskip. The page geometry now reserves the space instead (bottom margin 30 mm, footskip 15 mm), and the footer contains no negative skips.
3. **The software-default flag was unreliable.** A global boolean reset at shipout marked the wrong pages for multi-page tables. `\sdstar` now writes its page number to the `.aux` file; on the next pass the footer prints the note exactly on the pages that carry a marked value.

5.3 Page-bottom thresholds (`styles.py`, `\rptneedspace`)

The original preamble used `\needspace{5\baselineskip}`. `\needspace` leaves a `\penalty-100` behind. With `\raggedbottom` this made TeX prefer the point *before* a long table over the breaks between its rows, which produced a stray "Continued on next page" line and a doubled header. It was replaced by a penalty-free guard:

```
\newcommand{\rptneedspace}[1]{\par\begingroup
  \dimen@\pagegoal \advance\dimen@-\pagetotal
  \ifdim\dimen@<\dimexpr#1\relax\ifdim\dimen@>\z@\vfil\break\fi\fi
  \endgroup}
```

It is applied (with lengths from `SPACING`) before every table (9 lines), figure (12 lines), section heading (14 lines, so a heading is never left alone at the foot of a page) and before a heading that is followed directly by a figure (0.48 of the text height).

5.4 Short tables, notes, widows

- **Short tables are kept whole.** The builder estimates the height of a table from its rows, the column widths and the text length of the cells. If the estimate is at most half the text height, `\rptneedspace{<estimate>}` is emitted first, so the table moves to the next page instead of being split. The estimate only has to be roughly right: if it is too low the table simply breaks between rows, with repeated headers.
- **Notes stay with their table.** A note used to be a paragraph after the table and could be separated from it (or split in two). It is now the last, un-ruled row of the table and is bound to the row above it.
- **Widow/orphan control.** At least two data rows stay together below the header and before the end of a broken table.

5.5 Table width and captions

Column widths are fractions of `\rptTableWidth`, which the builder sets for every table to the line width minus cell paddings and rules. All tables therefore have exactly the text width (the original mixed 14 cm, 15.5 cm and 16.3 cm tables, some centred, some left-aligned). Captions are numbered automatically per chapter ("Table 5.3"); tables and figures of the un-numbered executive summary are numbered "ES.1", "ES.2" instead of the hand-written "Table 1 / Figure 1".

5.6 Live-load tables (`live_loads.py`, `chap3.py`)

`selected_vehicle_rows()` walks the eight IRC vehicles in the order of the Loading tab and keeps those whose check box key is set (plus user-defined vehicles). For each it reports the total load of one vehicle/train (IRC 6 Cl. 204), the impact factor $1+I$ for steel bridges (Cl. 208), and the braking load: considered or not (Class SV has its own opt-in key), value = 20% of the vehicle load (Cl. 211.2) and the eccentricity above the deck (Cl. 211.3, user value or 1.2 m). A second table gives the governing design braking force and the centrifugal force (Cl. 212; not applicable to the straight alignment OsdagBridge supports), a third the footpath live load (Cl. 206.1). These values are reported for information next to the inputs; the design engine is not affected.

5.7 Charts (`charts.py`)

The charts are drawn with matplotlib's object-oriented API on an Agg canvas — no `pyplot`, no global rcParams — because the desktop application builds the report in a background `QThread`. Each function writes a PNG into `<build dir>/images` and returns its path; the chapter embeds it through `figures.figure()` (PyLaTeX `Figure`) with a path relative to the build directory. The images disappear with the temporary directory after compilation.

`design_summary.collect()` builds one list of governing checks (element, check, combination, demand, capacity, UR). The same list feeds the *Overall Design Check Summary* table, both UR charts and the *Key Design Outcomes* table of the executive summary, so they always agree. The UR = 1.0 line, its colour, dash pattern and label come from `styles.CHART`.

5.8 Document assembly (`report_generator.py`)

The preamble is no longer a string template. `new_document()` builds a `pylatex.Document`:

- class, paper, fonts and geometry from `styles.document_arguments()`,
- the package list from `styles.latex_packages()`,
- the style preamble from `styles.preamble_tex()`.

`build_document()` appends the title page, the table of contents and the chapters. The existing compile step (bundled pdflatex discovered through `osdag_latex_env`, temporary build directory) is kept; the number of passes was raised from two to three because longtable widths, page references and the footer marker each need the `.aux` file of the previous pass. Only packages that are part of Osdag's bundled LaTeX distribution are used.

The public interface (`ReportRequest`, `ReportPayload`, `build_report_payload()`, `generate_report()`) is unchanged, so the desktop application needed no modification.

6 Rationale for the layout decisions

- **One builder instead of 60 templates.** Header repetition, keep-together rules and widths are properties every table must have. Implementing them once makes them impossible to forget in a new table.
- **Relative column widths.** Absolute widths were the reason tables had different widths and overflowed when the margins changed. Weights survive any change of the page geometry.
- **Unbreakable groups rather than smaller groups.** The largest group (13 rows) is about a third of a page, so keeping groups whole costs little space and removes the footer defect at its source.
- **Reserve footer space in the geometry.** A footer that needs negative skips to fit will collide with content sooner or later; a larger bottom margin is robust.
- **Left-aligned text, centred numbers, header fill in a brand tint.** Text columns are ragged-right (no stretched word gaps in narrow cells), numeric columns are centred, and the header row is visually distinct

without heavy rules. Many-column tables switch to a compact font and padding through a flag of their layout.
- **Status as meaning, not markup.** Chapters call `pass_fail()` / `ratio_tex()`; how PASS, FAIL and an exceeded ratio look is decided in one place.
- **Charts follow the table.** Charts are placed directly after the table they summarise and use the element colours consistently in both UR charts and the tonnage chart.

7 Content changes

The refactoring is about presentation, but a few defects in the content were fixed on the way:

- *Reactions at Supports* (Table 4.2) was an empty three-row table; it now lists the stored reactions. Load cases are sorted (basic cases, combinations, envelopes).
- Girder plate dimensions were printed as "1.67 mm" because the optimiser writes the section keys back in metres; `dimension_mm()` converts them. Floating-point noise such as `11.399999999999999` is trimmed.
- The executive summary stated "Girder design pass . . . " as fixed text; the outcome table now reflects the actual results (the sample design fails in fatigue, UR 1.87).
- The Section 5.5 summary gained the transverse-shear row from stored values and a Status column; the end-diaphragm rows are labelled compression/tension (they were labelled moment/shear but showed bracing forces).
- End diaphragms: when the analysis stores no separate end-diaphragm member forces (as in the sample design), the report — like upstream — shows the governing cross-bracing member design for a bracing-type diaphragm. This is now stated in a note under the tables instead of being silent.
- The bill of materials has a row for the end diaphragms.

8 Verification

- **Unit tests:** `pytest tests/unit/test_report_layout.py` — 29 tests: header repetition (one- and two-level), unbreakable groups, notes, table widths, footer geometry, absence of formatting in chapter files, layout names, live-load data, chart files and the threshold line, design summary, assembly of the whole document.
- **LaTeX log:** the after-report compiles without errors; 60 pages.
- **Footer clearance, measured:** every page of both PDFs was rasterised and the distance between the lowest content and the footer rule was measured. Before: 1.6 mm at the worst page, below 5 mm on six pages. After: at least 11.8 mm on every page (the two pages carrying the "* Software default value" footer note excepted — the note is part of the footer).
- **Visual check:** every page of the after-report was rendered and inspected: all continued tables carry caption and header, no table or label touches the footer, no heading is left at the bottom of a page.
- **Same data:** values were spot-checked against the before-report (e.g. deflections, fatigue ratios, quantities).

9 Reproducing the reports

```
# real GUI run: location, span, Design, Generate Report (same arguments for before and after)
python tools/report_demo.py --out out/Report_After.pdf --district Pune \
       --designer "Vivek Manik Warkad" --payload out/payload.pkl

# fast re-render of the stored design (no GUI, no analysis) while working on the layout
python tools/render_report.py out/payload.pkl --out out/Report_After.pdf

pytest tests/unit/test_report_layout.py
```

Environment note. Development was done on macOS (Apple silicon). `environment.yml` cannot be solved there as it is, because the `osdag` conda package depends on `osdag_latex_env`, which is published for Linux and Windows only. The environment was created from the same channels with `osdag` installed without that dependency, and TinyTeX provided `pdflatex`. The report code itself is platform independent and uses only LaTeX packages

contained in `osdag_latex_env`.

10 Known limitations

- The height estimate for short tables is a heuristic based on text length; a table with unusually tall cells may still be split (it then breaks cleanly with repeated headers).
- Wind force components and the number of design lanes are shown as "—" when the design run does not store them; this is unchanged from upstream.
- Per-girder tables repeat identical values for every girder because the design engine stores one governing section; the structure was kept so that girder-specific results can be filled in later without touching the layout.